DATA ENGINEERING COURSE

Python Modules & Code Organisation

Assignment

Instructor: Tanishq Tyagi

Topics Covered in This Assignment

- Creating and using custom Python modules
- Structuring functions across multiple files
- Applying import techniques in a real program

Student Instructions

- Read each question carefully before beginning your solution.
- Write clean, well-structured Python code following PEP 8 conventions.
- Use meaningful variable and function names throughout.
- Add appropriate comments and docstrings to your code.
- Each program must be split across at least two files: a utility module and a main program file.
- Test your program with multiple inputs before submitting.
- Submit all .py files as part of your solution.

■ Important Note

This assignment is designed to be attempted independently. Focus on applying the module concepts taught in class: creating utility modules, importing functions, and using the if __name__ == "__main__" guard correctly.

Assignment Questions

Choose ONE of the following two project options and implement it completely.

Option A — Modular Calculator

Q1.

Create a utility module named `math_util.py` containing all basic mathematical operation functions. Then create a separate main program named `calculator.py` that imports from `math_util` and implements a user-driven calculator.

Requirements for math_util.py:

- Define an `add(a, b)` function that returns the sum of two numbers.
- Define a `subtract(a, b)` function that returns the difference.
- Define a `multiply(a, b)` function that returns the product.
- Define a `divide(a, b)` function that returns the quotient. Handle division by zero appropriately.
- Include a module-level docstring describing the module's purpose.
- Include docstrings for each function.

Requirements for calculator.py:

- Import `math_util` using any of the three import methods taught in class (justify your choice in a comment).
- Accept two numbers and an operation from the user as input.
- Display the result of the chosen operation.
- Handle invalid operations gracefully with an appropriate message.
- Use the `if __name__ == "__main__"` guard correctly.
- Allow the user to perform multiple calculations without restarting the program (implement a loop that continues until the user chooses to exit).

Expected Behaviour:

```
=== Python Calculator ===
Enter first number  : 10
Enter second number : 4
Enter operation (+, -, *, /, quit): /
Result: 2.5

Enter first number  : 7
Enter second number : 0
Enter operation (+, -, *, /, quit): /
Error: Cannot divide by zero.

Enter first number  : quit
Exiting calculator. Goodbye!
```

Option B — String Analyser

Q1.

Create a utility module named `string_util.py` containing all string analysis functions. Then create a separate main program named `string_analyser.py` that imports from `string_util` and implements a user-driven string analyser.

Requirements for string_util.py:

- Define a `count_words(text)` function that returns the number of words.
- Define a `count_vowels(text)` function that returns the number of vowels.
- Define a `reverse_string(text)` function that returns the reversed string.
- Define an `is_palindrome(text)` function that returns True if the string is a palindrome (case-insensitive, ignoring spaces).
- Define a `most_common_char(text)` function that returns the most frequently occurring character in the string.
- Include a module-level docstring and docstrings for each function.

Requirements for string_analyser.py:

- Import `string_util` using any of the three import methods taught in class (justify your choice in a comment).
- Accept a string from the user as input.
- Display all five analysis results in a well-formatted output.
- Use the `if __name__ == "__main__"` guard correctly.
- Allow the user to analyse multiple strings without restarting the program.

Expected Behaviour:

```
=== String Analyser ===
Enter a string (or 'quit' to exit): racecar
```

```
Analysis Results:
  Word count      : 1
  Vowel count     : 3
  Reversed        : racecar
  Is palindrome   : True
  Most common ch  : r

Enter a string (or 'quit' to exit): Hello World

Analysis Results:
  Word count      : 2
  Vowel count     : 3
  Reversed        : dlroW olleH
  Is palindrome   : False
  Most common ch  : l

Enter a string (or 'quit' to exit): quit
Exiting String Analyser. Goodbye!
```

Common Requirements (Both Options)

- Your solution must consist of at least two separate Python files.
- The utility module must not contain any user input or print statements — only function definitions.
- The main program must use the `if __name__ == "__main__"` guard.
- Choose your import method deliberately and add a one-line comment explaining your choice.
- Follow PEP 8 coding style: proper indentation, spacing, and naming conventions.
- Handle edge cases (e.g., empty input, invalid operations, division by zero) gracefully.

Submission Guidelines

Submit all .py files for your chosen option. Ensure your files are named exactly as specified. Your code must run without errors before submission.